



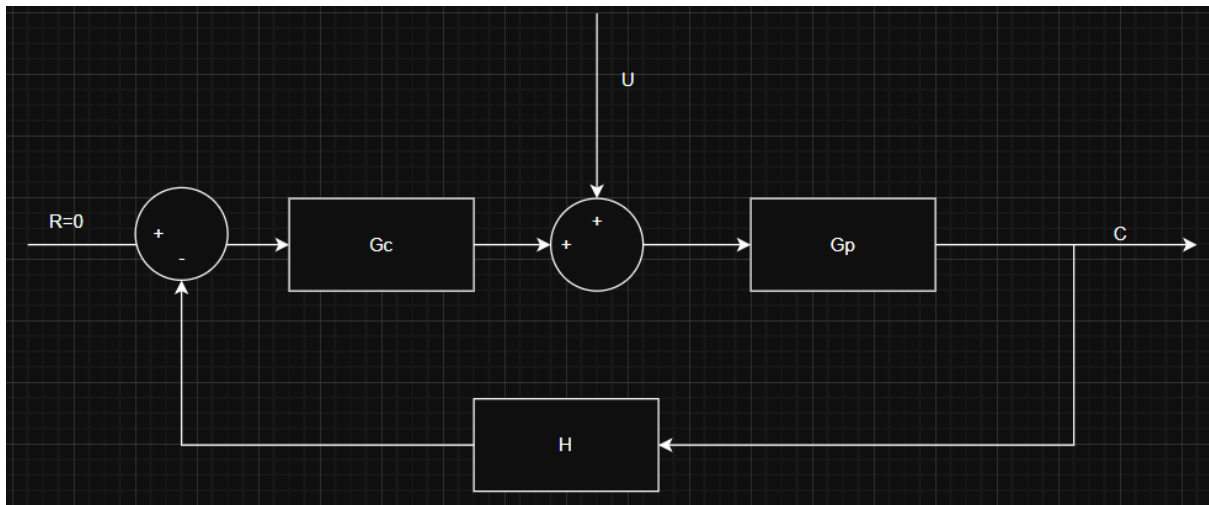
Tecnologías para la Automatización

Final 12/03/2026

Capriotti Santiago 51810

Carrera: ISI

Diagrama de bloques y clasificación del sistema



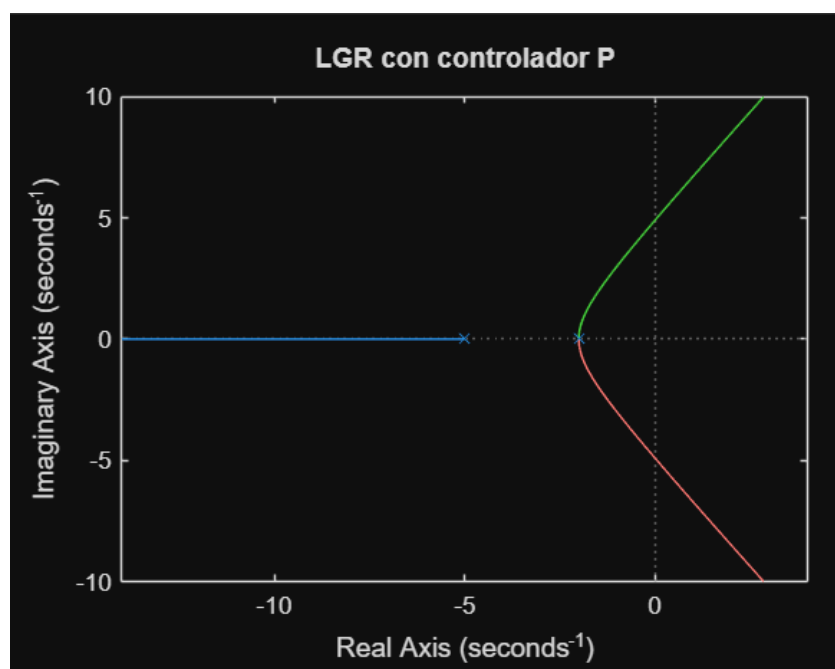
Nuestro sistema es **regulador** debido a que nos piden "...mantener la presión de vapor constante en un colector principal, a pesar de las variaciones de demanda...". Esto significa que debemos mantener la salida constante a pesar de las perturbaciones que puedan presentarse. Por esto, consideramos $R=0$ y asumimos una realimentación unitaria $H=1$.

Reconstrucción de la FT

Dada la **Figura 1** que nos dan, podemos observar que tenemos tres polos, uno en -5 y otros dos superpuestos en -2 . Dado que no se observan ceros y asumimos un controlador proporcional, podemos decir que nuestra FTLA estará definida como:

$$FTLA = K_p / ((s+2)*(s+2)*(s+5))$$

Tomando como valor del $K_p=1$ obtenemos la siguiente salida.



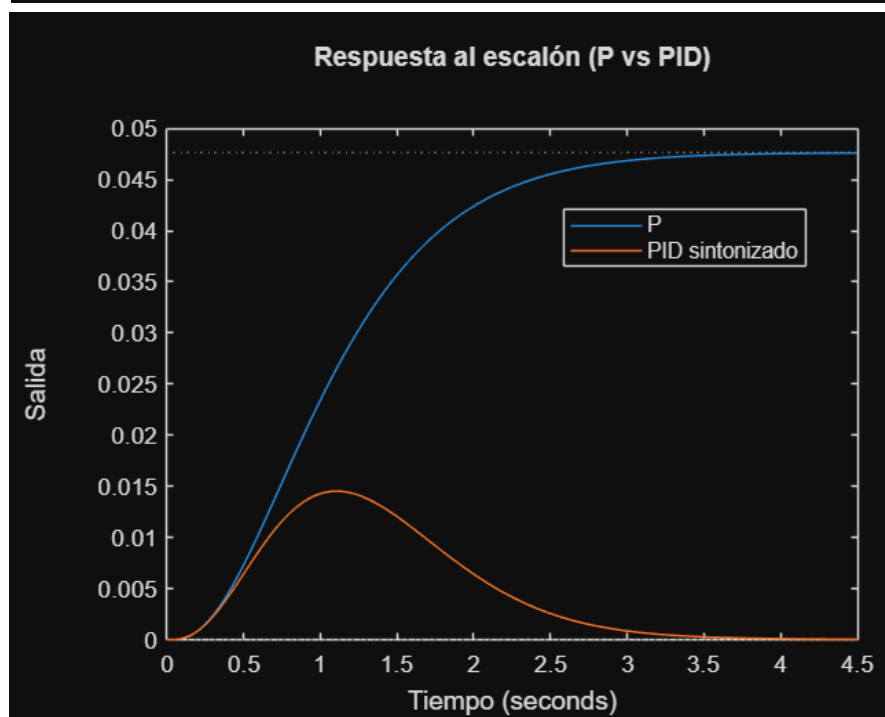
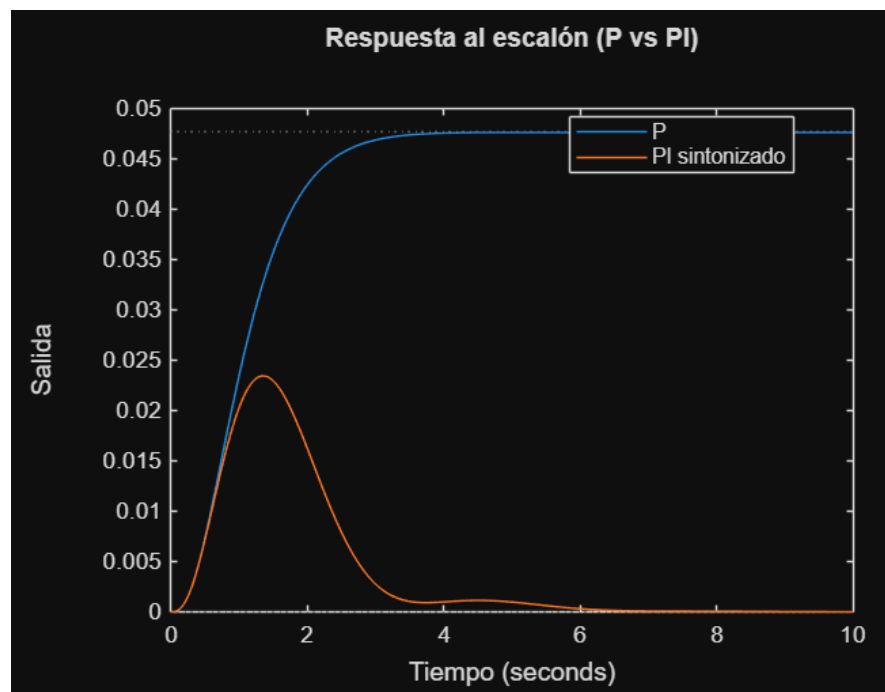
Podemos ver que obtenemos exactamente la misma salida que en la Figura 1, lo que nos da certeza de que nuestra FTLA es correcta.

Estrategia de control

Se nos pide que elijamos el controlador MÍNIMO que cumpla con los siguientes requisitos:

- Error en estado estacionario cero ante entrada de tipo escalón en la presión.
- Respuesta rápida ante cambios de consumo de vapor.

El hecho de que nos pidan eliminar por completo el error en estado estacionario y a su vez tener una respuesta rápida ante cambios nos hace pensar que un controlador PI o PID son las mejores opciones. Para tomar una decisión vamos a analizar las respuestas al escalón, como salidas en MATLAB:



En las imagenes anteriores se puede ver la comparativa de una salida ante una entrada escalón de un controlador P vs PI y de un P vs PID. Podemos ver que en ambos se logra eliminar el error en estado estacionario en forma correcta, sin embargo, vemos que el controlador PID, al reducir notablemente el sobreimpulso, logra que llegue a su estado estacionario casi en la mitad del tiempo que el controlador PI, por lo que concluimos que la opción que mejor se adapta a las condiciones establecidas es el **controlador PID**.

Código MATLAB

PID

```
clear; clc; close all;
```

```
s = tf('s');
```

```
Gp = 1/((s+2)*(s+2)*(s+5)) ;
```

```
H = 1 ;
```

```
Kp_P = 1;
```

```
GcP = Kp_P;
```

```
L_P = GcP * Gp * H;
```

```
figure(1);
```

```
rlocus(L_P); grid off;
```

```
title('LGR con controlador P');
```

```
% Sintonizamos el PID con pidtune
```

```
[GcPID, infoPID] = pidtune(Gp * H, 'PID');
```

```
% Mostramos los parámetros sintonizados
```

```
Kp_PID = GcPID.Kp;
```

```
Ki_PID = GcPID.Ki;
```

```
Kd_PID = GcPID.Kd;
```

```
fprintf('PID sintonizado:\nKp = %.3f, Ki = %.3f, Kd = %.3f\n', Kp_PID, Ki_PID, Kd_PID);
```

```
T_reg_P = minreal( Gp / (1 + GcP * Gp * H) );
```

```
T_reg_PID = minreal( Gp / (1 + GcPID * Gp * H) );
```

```
figure(2);
```

```
step(T_reg_P, T_reg_PID);
```

```
grid off;
```

```
legend('P', 'PID sintonizado', 'Location', 'best');
```

```
title('Respuesta al escalón (P vs PID)');
```

```
xlabel('Tiempo'); ylabel('Salida');
```

PI

```
clear; clc; close all;
```

```
s = tf('s');
```

```
Gp = 1/((s+2)*(s+2)*(s+5))
```

```
H = 1;
```

```
Kp_P = 1;
```

```
GcP = Kp_P;
```

```
L_P = GcP * Gp * H;
```

```
% Sintonizamos el PI
```

```
[GcPI, infoPI] = pidtune(Gp * H, 'PI');
```

```
% Parámetros sintonizados
```

```
Kp_PI = GcPI.Kp;
```

```
Ki_PI = GcPI.Ki;
```

```
fprintf('PI sintonizado:\nKp = %.3f, Ki = %.3f\n', Kp_PI, Ki_PI);
```

```
T_reg_P = minreal( Gp / (1 + GcP * Gp * H) );
```

```
T_reg_PI = minreal( Gp / (1 + GcPI * Gp * H) );
```

```
figure(1);
```

```
step(T_reg_P, T_reg_PI);
```

```
grid off;
```

```
legend('P', 'PI sintonizado', 'Location', 'best');
```

```
title('Respuesta al escalón (P vs PI)');
```

```
xlabel('Tiempo'); ylabel('Salida');
```